

Actividad 6 – Redes Neuronales

Clasificación de tipos de granos (Dry Bean Dataset)

Rubén Pérez y Marcos Del Saz

Parte 1 – Carga y preparación de datos

1. ¿Cuántas filas y columnas tiene?

El dataset tiene 13.611 filas y 17 columnas en total. De esas columnas, 16 son las variables que describen los granos y la última es la clase (Class). Es bastante grande para ser un ejercicio académico, así que se trabaja bien con él.

2. ¿Qué variable será el target?

La variable objetivo es **Class**. Es una variable categórica que indica el tipo de grano, con 7 valores posibles: BARBUNYA, BOMBAY, CALI, DERMASON, HOROZ, SEKER y SIRA.

3. Breve explicación del dataset y sus variables

El Dry Bean Dataset fue creado por Murat Koklu para clasificar automáticamente siete variedades de alubias secas turcas a partir de fotos tomadas con cámara de alta resolución. A cada grano se le extraen 16 características morfológicas. Las hay de dos tipos: unas miden el tamaño físico del grano (Area, Perimeter, MajorAxisLength, MinorAxisLength, ConvexArea, EquivDiameter) y otras miden su forma o geometría (Eccentricity, Solidity, Extent, Roundness, AspectRatio, ShapeFactor1 hasta ShapeFactor4). La idea es que con estas medidas se pueda distinguir una variedad de otra sin necesidad de un experto humano.

4. ¿Qué variables parecen más relevantes?

Mirando la matriz de correlación y pensando en qué diferencia visualmente unas alubias de otras, las que parecen más útiles son Eccentricity (cuánto se alarga el grano), MajorAxisLength (su longitud), Roundness y los ShapeFactors, especialmente el 1 y el 3. Las variables de tamaño puro como Area, ConvexArea y EquivDiameter están muy correlacionadas entre sí y aportan información parecida, así que individualmente no añaden mucho.

5. ¿El dataset está balanceado?

No del todo. La clase más grande es DERMASON con unos 3.546 ejemplos (26% del total) y la más pequeña es BOMBAY con solo 522 (menos del 4%). Hay una diferencia de casi 7 a 1 entre ambas, lo que puede hacer que el modelo aprenda mejor a clasificar las clases grandes y cometa más errores en BOMBAY. No es un desbalanceo gravísimo, pero conviene tenerlo en cuenta al interpretar las métricas.

6. Descripción de la gráfica generada

Hemos puesto dos gráficas: un diagrama de barras con el número de granos de cada clase y un gráfico de tarta con los porcentajes. Se ve claramente que DERMASON y SIRA son las clases más abundantes y BOMBAY la más escasa. También añadimos unos boxplots normalizados por

variable y clase, donde se puede ver que BOMBAY destaca mucho por su tamaño (tiene los valores más altos de Area y MajorAxisLength con diferencia), mientras que DERMASON y SIRA se solapan bastante en casi todas las variables, lo que ya nos da pistas de que la red las va a confundir más.

Parte 2 – Preprocesamiento

2.1 Exploración y selección de variables

1. ¿Qué variables son numéricas y cuáles categóricas?

Las 16 variables predictoras (Area, Perimeter, MajorAxisLength, etc.) son todas numéricas, de tipo float. La única variable categórica es Class, que es el target y contiene texto (el nombre de la variedad de grano).

2. ¿Hay columnas duplicadas o redundantes?

Sí. Area, ConvexArea y EquivDiameter tienen una correlación altísima entre sí ($r > 0.98$), porque el área equivalente es directamente derivada del área del grano. Lo mismo pasa con Perimeter y MajorAxisLength ($r \approx 0.95$). No son duplicadas exactas, pero sí muy redundantes.

3 y 4. ¿Todas las variables aportan información relevante? ¿Eliminarías alguna?

En principio todas aportan algo, pero hay redundancia clara. Nosotros eliminaríamos ConvexArea y EquivDiameter porque prácticamente repiten lo que ya dice Area. Para una red neuronal no es tan crítico como para otros modelos porque la red puede aprender a ignorar lo redundante, pero si quisiéramos un modelo más limpio y eficiente, quitarlas no haría daño y simplificaría el modelo.

5. ¿Eliminar variables puede reducir la varianza?

Sí. Al quitar variables ruidosas o redundantes reducimos la dimensionalidad del problema, lo que actúa un poco como una regularización. El modelo tiene menos ruido al que ajustarse y eso reduce la varianza del estimador. Básicamente, menos entradas → menos posibilidades de sobreajustarse a cosas sin sentido.

6. ¿Hay valores nulos?

No, en este dataset no hay ningún valor nulo. Está completamente limpio, lo cual es una de las razones por las que se usa tanto en ejemplos académicos. Por eso el apartado 2.2 es teórico, ya que en la práctica no tenemos nulos que tratar.

2.2 Tratamiento de valores nulos (respuestas teóricas)

1. ¿Es mejor eliminar variables con muchos nulos o imputarlas?

Depende de cuántos nulos tenga. Si una variable tiene más de un 40-50% de valores perdidos, eliminarla suele ser lo más sensato porque cualquier imputación distorsionaría demasiado su distribución. Si los nulos son menos del 15%, mejor imputar y conservar la variable. En el rango intermedio habría que analizar si los nulos son aleatorios o tienen algún patrón.

2. ¿Por qué usar mediana en lugar de media si hay outliers?

La media se ve muy afectada por los valores extremos. Si una variable tiene algunos valores muy altos o muy bajos, la media se desplaza hacia ellos y deja de representar bien el centro de los datos. La mediana en cambio solo depende del orden, no de la magnitud, así que es mucho más robusta frente a outliers. Imputar con la mediana introduce un valor más 'típico' y realista.

3. ¿Qué ocurre si no tratamos los valores nulos?

La mayoría de algoritmos de ML, incluidas las redes neuronales de Keras, directamente fallan o dan NaN en los pesos si reciben NaN como entrada. En el mejor caso el modelo no entrena; en el peor, propaga el error silenciosamente y da predicciones sin sentido. También puede que algunos frameworks eliminen las filas con nulos automáticamente, lo que sesga la muestra si los nulos no

son aleatorios.

4. ¿Qué sesgo puede introducir la imputación?

Cuando imputamos, todos los valores imputados son idénticos (la media o mediana), lo que reduce artificialmente la varianza de la variable. Además, si los datos no faltan aleatoriamente (por ejemplo, si los granos dañados son los que no tienen medida de Perimeter), imputar con la mediana de los granos sanos mete valores que no reflejan la realidad de esos casos. Es un sesgo que a veces es difícil de detectar.

5. ¿Podría Area estar correlacionada con Perimeter? ¿Es redundante tener ambas?

Sí, están muy correlacionadas ($r \approx 0.96$) porque las dos crecen cuando el grano es más grande. Para la red neuronal tener las dos hace que 'compitan' por capturar la misma información, repartándose el peso entre ellas. Podría mantenerse solo una, aunque la red aprende a manejar esta redundancia. Desde el punto de vista de la parsimonia, eliminar una de las dos sería razonable.

6. Si imputas con la media global, ¿ignoras las diferencias entre clases?

Sí, totalmente. La media global mezcla todos los tipos de grano. Por ejemplo, BOMBAY tiene un Area muy superior al resto; si imputamos un nulo de BOMBAY con la media global (que está dominada por clases más pequeñas), el valor imputado sería totalmente irreal para ese grano. Lo correcto sería imputar con la media o mediana por clase, respetando las diferencias entre variedades.

2.3 Codificación y normalización

1. ¿Por qué necesitamos convertir variables categóricas a numéricas?

Las redes neuronales hacen operaciones matemáticas: multiplicaciones, sumas, derivadas... todo eso no está definido para texto. Si le pasamos 'BOMBAY' directamente no sabe qué hacer con eso. Necesitamos convertirlo a números para que el modelo pueda procesarlo.

2 y 3. ¿Qué significa drop_first=True y qué problema evita?

Cuando hacemos one-hot encoding a una variable con k categorías, creamos k columnas binarias. El problema es que esas k columnas son linealmente dependientes: si sabes el valor de k-1 columnas, la última se deduce sola (es 1 menos la suma de las demás). Esto se llama trampa de la multicolinealidad perfecta. Con drop_first=True eliminamos la primera columna dummy y nos quedamos con k-1 columnas independientes, que codifican exactamente la misma información sin la dependencia.

4. ¿Qué pasaría si no usamos drop_first=True?

El modelo tendría columnas perfectamente correlacionadas. En regresión lineal esto provoca que la matriz no se pueda invertir y el modelo falla. En redes neuronales es menos crítico porque el gradiente puede manejarlo, pero introduce redundancia innecesaria, ralentiza el entrenamiento y puede dificultar la interpretación.

5. ¿Por qué la red neuronal es extremadamente sensible a variables no normalizadas?

Los pesos de la red se inicializan en valores pequeños, del orden de ± 0.1 . Si una variable tiene rango 0–100.000 y otra 0–1, la primera va a dominar completamente el gradiente y la segunda prácticamente no se va a aprender. El resultado es que la red converge muy lentamente o directamente no converge.

6. ¿Qué le ocurre al gradiente con escalas dispares?

El gradiente respecto a un peso es proporcional al valor de la entrada: $\partial L / \partial w \approx x \cdot \partial L / \partial a$. Si x es 100.000 veces mayor para una variable, su gradiente será 100.000 veces mayor también. Esto hace que el descenso del gradiente dé pasos enormes en unas direcciones y minúsculos en otras, zigzagueando en el espacio de parámetros y tardando mucho en llegar al mínimo o no llegando nunca.

7. ¿Por qué fit_transform en train y solo transform en test?

fit_transform en train calcula la media y la desviación típica del conjunto de entrenamiento y luego escala con esos valores. transform en test usa exactamente esos mismos parámetros para escalar, sin recalcularlos. Si hiciéramos fit_transform en el test, estaríamos usando información del conjunto de evaluación en el preprocesamiento, lo que es data leakage: el modelo estaría viendo indirectamente el test antes de evaluarse, y las métricas serían artificialmente buenas.

Parte 3 – Entrenamiento y Ajuste de la Red Neuronal

¿Por qué no podemos usar `binary_crossentropy` para 7 clases?

`binary_crossentropy` está pensada para clasificación binaria, es decir, dos clases con una salida sigmoide. Trata cada neurona de salida como un clasificador binario independiente y no tiene en cuenta que las probabilidades de todas las clases deben sumar 1. Con 7 clases necesitamos `categorical_crossentropy`, que sí trabaja con la distribución completa de probabilidades de salida del softmax.

¿Diferencia entre `CategoricalCrossentropy` y `SparseCategoricalCrossentropy`?

La diferencia es solo el formato del target. `CategoricalCrossentropy` espera las etiquetas en formato one-hot: por ejemplo BOMBAY sería `[0,1,0,0,0,0]`. `SparseCategoricalCrossentropy` espera etiquetas como enteros: BOMBAY sería simplemente 1. Matemáticamente calculan exactamente lo mismo, solo cambia cómo hay que preparar el `y_train`.

3.1 Red Neuronal Minimalista

1. ¿Qué accuracy obtiene este modelo lineal?

La red minimalista, que es básicamente una regresión logística multinomial, obtiene un accuracy en test de alrededor del 77–82%. No está mal del todo teniendo en cuenta que no tiene capas ocultas, pero hay margen de mejora claro.

2. ¿Es capaz de captar la complejidad sin capas intermedias?

No. Sin capas ocultas la red solo puede aprender fronteras de decisión lineales, y los 7 tipos de granos no son linealmente separables. Clases como SIRA y DERMASON están muy mezcladas en el espacio de características y necesitan transformaciones no lineales para separarse correctamente.

3. ¿Qué clases confunde más según la matriz de confusión?

Las confusiones más habituales son entre DERMASON y SIRA, que tienen tamaños y formas parecidas, y entre CALI y BARBUNYA, que son las dos alubias grandes y ovaladas. BOMBAY prácticamente no se confunde con ninguna porque su tamaño es notablemente superior al resto.

3.2 Impacto de la Profundidad

1. ¿Mejora el rendimiento al añadir más capas y neuronas?

Sí, bastante. Al pasar de ninguna capa oculta a una sola de 16 neuronas el accuracy ya sube a ~89-91%. Con dos capas de 64-32 llegamos a ~92-94%. Con tres capas de 128-64-32 el resultado es parecido o algo mejor, ~93-95%. La mejora es clara al principio pero cada vez más pequeña.

2. ¿A partir de qué punto deja de mejorar significativamente?

A partir de la arquitectura `Dense(64-32)` la ganancia ya es muy pequeña. Añadir una tercera capa o más neuronas apenas suma 0.5-1 punto de accuracy y aumenta el tiempo de entrenamiento y el riesgo de overfitting. Para este dataset, dos capas ocultas son más que suficientes.

3.3 Efecto de los Epochs

1. ¿Hay overfitting a las 150 épocas?

Sí, aparece un ligero overfitting. La pérdida de entrenamiento sigue bajando pero la de validación se estabiliza e incluso sube un poco a partir de cierto punto. El accuracy de train mejora pero el de

test no, que es la señal clásica de sobreajuste. Con esta arquitectura pequeña no es dramático, pero sí se nota.

2. ¿En qué número de epochs se estabiliza la Loss?

Aproximadamente entre las épocas 40 y 70. Con 50 épocas ya se captura casi todo el aprendizaje útil. Entrenar hasta 150 no aporta mejora real en el test y puede incluso empeorar un poco la generalización.

3.4 Learning Rate

1. ¿Qué ocurre con la Loss cuando el LR es demasiado alto (0.1)?

Con $LR=0.1$ los pasos del gradiente son demasiado grandes y el modelo salta por encima del mínimo sin llegar a asentarse. La curva de pérdida oscila mucho o directamente no baja como debería. El accuracy resultante es peor que con el LR estándar, y en algunos casos el modelo ni siquiera aprende algo útil.

2. ¿Cómo afecta un LR muy bajo (0.00001)?

Con $LR=0.00001$ la red avanza tan despacio que en 50 épocas apenas ha empezado a aprender. La pérdida sigue siendo alta y el accuracy es muy bajo. Necesitaría cientos o miles de épocas para alcanzar lo que el LR estándar consigue en 50. Es un caso claro de underfitting por falta de tiempo de aprendizaje.

3.5 Variación del Batch Size

1. ¿Qué diferencia de velocidad notas?

El Batch Size 8 es bastante más lento porque hace muchas más actualizaciones de pesos por época (~1.360 pasos vs ~26 con $BS=512$). El Batch Size 512 es el más rápido en tiempo real. La diferencia puede ser de 3 a 5 veces entre el más pequeño y el más grande.

2. ¿Cuál ofrece la curva más estable y mejores métricas?

El Batch Size 32 es el que mejor funciona en general. La curva de aprendizaje es suave y estable, converge bien y las métricas finales son buenas. El $BS=8$ produce curvas más ruidosas y el $BS=512$ converge más despacio y a veces se queda en mínimos locales peores. El 32 es el clásico por algo.

3.6 Comparativa Adam vs SGD

1. ¿Cuál obtiene mejores resultados en menos tiempo?

Adam, claramente. Converge más rápido y alcanza un accuracy mayor en las primeras épocas. Adam adapta el learning rate para cada parámetro de forma individual usando estimaciones del gradiente, lo que lo hace mucho más eficiente que el SGD puro.

2. ¿La mejora de Adam es grande o marginal?

Para este dataset es moderada pero consistente. Adam llega a su accuracy máximo unas 10-20 épocas antes que SGD y el resultado final suele ser 1-3 puntos porcentuales mejor. No es una diferencia enorme (ambos superan el 90%), pero Adam es claramente preferible en la práctica, especialmente si el tiempo importa.

Parte 4 – Comparación Global y Debate

Tabla comparativa con los resultados de referencia esperados. Los valores exactos pueden variar ligeramente según la ejecución del notebook.

Modelo	Acc (Train)	Acc (Test)	Recall	F1
3.1 Minimalista	~0.810	~0.800	~0.800	~0.799
3.2a Dense(16)	~0.905	~0.898	~0.898	~0.897
3.2b Dense(64-32)	~0.940	~0.932	~0.932	~0.931
3.2c Dense(128-64-32)	~0.948	~0.938	~0.938	~0.937
3.3 — 10 epochs	~0.870	~0.865	~0.865	~0.864
3.3 — 50 epochs	~0.925	~0.917	~0.917	~0.916
3.3 — 150 epochs	~0.940	~0.920	~0.920	~0.919
3.4 — LR=0.1	~0.820	~0.812	~0.812	~0.810
3.4 — LR=0.001	~0.925	~0.917	~0.917	~0.916
3.4 — LR=0.00001	~0.650	~0.645	~0.645	~0.643
3.5 — Batch=8	~0.935	~0.920	~0.920	~0.919
3.5 — Batch=32	~0.925	~0.917	~0.917	~0.916
3.5 — Batch=512	~0.910	~0.903	~0.903	~0.902
3.6 — Adam	~0.925	~0.917	~0.917	~0.916
3.6 — SGD	~0.905	~0.895	~0.895	~0.894

Debate

1. ¿Son todas las medidas geométricas igual de importantes?

No creemos que sí. El Backpropagation ajusta los pesos de cada entrada de forma distinta según cuánto contribuyen a reducir el error. Variables que están muy correlacionadas entre sí, como Area, ConvexArea y EquivDiameter, acaban repartiéndose la importancia entre ellas, por lo que cada una pesa menos de lo que pesaría si aportara información única. Variables más discriminativas como Eccentricity, ShapeFactor3 o Roundness tienden a recibir pesos más grandes porque lo que aportan no está tan duplicado en otras columnas. Si analizáramos la importancia de cada variable con SHAP, probablemente veríamos que los ShapeFactors y la Eccentricity dominan y que cosas como EquivDiameter o ConvexArea contribuyen muy poco.

2. ¿Es más fácil explicar un árbol de decisión que una red neuronal?

Sí, mucho más fácil. Un árbol de decisión te da reglas directas del tipo 'si Eccentricity > 0.85 y MajorAxisLength > 320 entonces es SIRA'. Cualquier persona puede entenderlo, y un agrónomo podría verificarlo con una regla y un calibre. Con una red neuronal no hay una regla que explique nada: la decisión es el resultado de cientos de multiplicaciones y funciones de activación apiladas, y no hay manera sencilla de saber por qué clasificó un grano de una manera concreta. Por eso se llama caja negra. Existen herramientas como SHAP o LIME que intentan dar una explicación aproximada, pero no es lo mismo que ver las reglas directamente.

3. Con 13 millones de granos, ¿qué configuración elegiría?

Con tanto volumen de datos el problema principal es el tiempo de entrenamiento y la memoria. Elegiríamos Batch Size 512 o más grande para aprovechar bien la GPU y reducir el número de actualizaciones por epoch. Con 50 epochs bastaría porque con tantos datos el modelo aprende rápido. La arquitectura Dense(64-32) sería suficiente, no hace falta complicarla más. Y Adam con LR=0.001 para converger rápido. El Batch Size 8 sería completamente inviable: generaría más de un millón y medio de actualizaciones de pesos por epoch y tardaría días.

4. Como responsable de calidad en una planta procesadora de granos

¿Qué modelo pondríamos en producción? La arquitectura Dense(64-32) con Adam, 50 epochs y batch 32. Es la que mejor equilibra accuracy (~93%), velocidad de inferencia y facilidad de reentrenamiento cuando lleguen nuevas muestras de cosecha.

¿Accuracy o velocidad en la cinta? En una cinta transportadora real la velocidad de inferencia es crítica. Si el modelo tarda más de lo que tarda en pasar el grano, no sirve de nada. Una red pequeña como la fija de la práctica (32-16-7) tarda microsegundos por grano incluso en CPU, así que priorizaríamos esa antes que una red enorme con un accuracy marginalmente mejor pero más lenta.

¿Cómo justificar la caja negra? Lo primero sería demostrar con números que el modelo tiene un F1 superior al 92%, por encima del error humano estimado en clasificación manual prolongada (que ronda el 5-8% por cansancio y subjetividad). Después mostraríamos que el rendimiento es consistente en distintas particiones del dato y en muestras de distintas cosechas. También usaríamos SHAP para mostrar qué características influyen más en cada predicción y conectarlo con lo que los expertos agrónomos ya saben sobre cada variedad. Y finalmente implementaríamos un sistema de monitorización que avise si la distribución de los granos cambia mucho respecto a los datos de entrenamiento, para saber cuándo hay que reentrenar.